

HS300 drive

Complete maintenance manual

22 kW IGBT inverter, sensorless field-oriented control, driving a submersible pump from PV, battery or a weak grid.

Contents

- Manual
- Troubleshooting
- Fault Codes
- Comms

HS300 drive — maintenance manual

A 22 kW IGBT inverter driving a submersible pump from a PV array, from a battery bus, or from a weak grid, depending on the build. Sensorless field-oriented control with its own auto-tune; no encoder, nothing down the well but the motor.

What you need in front of you: a laptop with this folder, a USB-RS485 adapter (or the ST-Link's USB serial), and a multimeter.

Tell the scripts where the drive is — once

```
PORT.bat
```

Double-click it. It lists the serial ports it can see, asks which one the drive is on and which cable you are using, and makes the setting for you. Run it again to check or change it.

FLASH.bat, CHECK_SENSOR.bat, ZERO_ADC.bat and DASHBOARD.bat then all find the drive on their own and each says which setting it obeyed. If you start one before setting a port, it asks you there and then - you are never left stuck. A port typed on the command line always wins.

The setting is an empty file whose **name** is the whole thing:

```
tty4.485      COM4 over the USB-RS485 adapter
tty5.ttl      COM5 over the ST-Link USB serial / direct TTL
```

Keep one, not two - PORT.bat replaces the old one when you change it, because whichever is found first would otherwise win and the other is a trap. Put it here or one level up; both are searched.

It is tty, not com. Windows reserves COM1..COM9 as device names, over the whole name before the first dot, so File Explorer refuses com5.ttl as an invalid name. Some other tools create it anyway, which is worse rather than better - the file exists but Explorer may then refuse to rename or delete it. port5.ttl and ccom5.ttl also work, as does .rs485 for .485.

A setting whose name cannot be read as a port is **reported**, not ignored, and you are offered a replacement. One that cannot be understood must not look the same as none at all.

The dashboard

```
DASHBOARD.bat
```

Starts the agent and opens <http://localhost:8080>. Live telemetry, settings you can edit, and the commands. This is the easiest way to see what the drive is doing.

It holds the serial port while it runs. Nothing else can talk to the drive at the same time — close that window before FLASH.bat, CHECK_SENSOR.bat or ZERO_ADC.bat.

In order, on a drive you have not met before

1. CHECK_SENSOR.bat — the DC current sensor's range. Do this first, every time, on any board you have not personally checked. Reason in the next section.
2. ZERO_ADC.bat — only with the **DC bus at zero volts**. It measures the offsets as they sit, so whatever is present becomes "zero" for ever.
3. FLASH.bat <BOARD> <COM> — the proven image for that board.
4. DASHBOARD.bat — check the plate, then run a tune, then a test run.

Why CHECK_SENSOR comes first

New boards leave the factory with the DC current **offset pot at 2.5 V** and it must be turned down to about **0.5 V**. On a 3.3 V ADC a 2.5 V rest point leaves only about **27 A** of measurable current, and the sensor then simply

reports a flat maximum — no fault, no warning.

Because the MPPT verdict is $V_{dc} \times I_{dc}$, once the current rails the tracker sees power **fall** whenever it loads harder, so it parks on that false ceiling and reports it as the array's limit. One board ran that way for a whole morning with roughly a tenth of the array unused and nothing complaining.

CHECK_SENSOR.bat catches it in two minutes. If it says FAIL, turn the pot down until the rest point is near the bottom of the range, then ZERO_ADC.bat.

Settings worth knowing

Edit from the dashboard, or by name with the tools. **Over the wire they are in real units** — the panel's F-number and its $\times 10$ scaling are a different thing.

setting	what it is	watch out
PLATE_KW, PLATE_V_LL, PLATE_F_HZ, PLATE_I_RMS, PLATE_PP, PLATE_RPM	the motor nameplate	everything derives from this: the tune's current levels, the stall gate, i_open. A wrong plate is the most common cause of a tune that will not pass
F01 OVER_CURRENT	the trip	must be above what the pre-start itself draws, or the check trips
F03 OVER_VOLTAGE	bus OV trip	check it is reachable — on one board it sat above what the sensor could report
F05 OVER_TEMP	heatsink trip, 90 °C	59 °C at full load on the 37K is normal
F08 FMIN / F09 FMAX	frequency limits	FMAX is the plate frequency. Going past it over-speeds the pump
F10 ACC_FACTOR	ramp rate	
MODE	0 manual, 1 MPPT, 2 DC bus	change it on the fly. Stopping and restarting to change mode makes the tracker re-enter from the floor and the drive dives to minimum frequency
AUTOSTART_EN	starts on its own when the bus comes up	leave 0 unless you mean it. On a battery bus this once drained the pack flat overnight

Settings do not persist unless you save them

A write changes the live value only. **Without SAVE_SETTINGS it is lost at the next reset**, and these boards reset often. The dashboard's save button sends it; by hand it is CMD_SAVE_SETTINGS.

To **prove** a value is stored: save, then CMD_RELOAD_EEPROM, then read it back. A plain read-back straight after a write tells you nothing.

Normal readings, 22 kW well machine at full sun

f	48-50 Hz	Vdc	650-680 V
Idc	30-34 A	Pdc	20-22 kW
psi_r	0.985-0.99	iq	45-53 A peak
temp	55-60 C	BUS_SHED	1.000

BUS_SHED below 1.0 means the bus sagged and the drive softened the load. Brief dips on a cloud edge are the design working. Sitting at 0.2 is not.

Flashing

FLASH.bat <BOARD> <COM> uses the **STABLE** image for that board. It must print **both** of these:

```
verified and marked bootable
NEW app is up
```

If you do not see both, the drive did not take it.

- **An OTA that stalls part way needs SWD recovery, not another attempt.** Stop and ask the engineer.
- A **virgin board cannot be flashed over serial at all** — there is no bootloader yet. Use `full_<BOARD>_<CRC>.hex` with an ST-Link and **mass erase the chip first**, or stale metadata will make the bootloader refuse the app.
- A board with **no STABLE** has never been proven. Ask; do not flash LATEST.

Keep a record

`drive_backup.py` saves the EEPROM settings and the firmware CRC before you change anything. Do it before a tune or a flash — it is the only way to get a drive back to how it was.

Troubleshooting — symptom first

Every entry here is something that actually happened on this plant, with what it turned out to be. For codes, see `FAULT_CODES.md`.

The drive will not talk

symptom	cause	fix
no reply at all	wrong baud	the baud is per drive (BAUD_COM2); this plant's well drive is 9600 . Tools scan it now and say so
a tool says "no drive ... silent at every baud"	the drive, not the PC	the port opened, so it is not the laptop. Powered? A/B on COM2 , not COM1? Polarity?
no reply, port opens	the dashboard is running	it holds the port — close that window
Access is denied on the port	another tool still has it	close it; a killed shell can leave its Python child holding the port
port vanished entirely	the drive is unpowered	on a PV build the adapter dies with the bus. Check the array
reads work, numbers are nonsense	the PC map does not match the firmware	run <code>check_link_api.py</code> ; re-run <code>UPDATE.bat</code> to refresh tools/

It runs, but not as well as it should

symptom	cause	fix
frequency sits well below FMAX in good sun	the slip cap fed by a noisy rotor estimate	the rotor estimate swings ± 3 Hz; if the cap's input is lightly filtered its noise minima clamp the setpoint. Fixed in firmware by filtering harder — make sure the drive has a current image
power seems capped at a round number and will not rise	the DC current sensor is railing	run <code>CHECK_SENSOR.bat</code> . See <code>MANUAL.md</code> — the offset pot
tracker parks low and will not climb	wrong MPPT margin	F12 is the margin $\times 10$. A stale EEPROM value of 35 means margin 3.5, which parks far short
climbs, collapses, climbs again, about once a minute	the margin is too aggressive for that irradiance	the MPP of a constant-power load is a cliff , not a hill — overshoot does not cost a little power, it costs the run. Raise F12
water stops at dusk but no fault	LINGER (13)	correct behaviour — it was below pump cut-in and stopped fighting

A useful cross-check: compare `F0C_PEST` against $VDC \times IDC$. Motor power cannot exceed DC input power. If the ratio jumps above ~ 1.0 , the DC current reading is wrong — that is what exposed the railed sensor.

Starting

symptom	cause	fix
pre-start fails right after a stop	the motor is still coasting; back-EMF looks like a fault	wait ~ 2 s and start again
pre-start reports an open phase that is not open	relay or ADC-offset chain	cost a day on the grid board, 2026-09-19. Check the relay actually pulled in, and the offsets
asymmetry (31) that makes no sense	a probe vector aborted on over-current, leaving no measurement	check <code>PS_FLAGS</code> for <code>0x4000</code> first
ten UV trips in a row at dawn	not enough array to magnetise	expected on a weak morning. <code>START_DEFER</code> (12) now handles it

symptom	cause	fix
drive reads 25 V on a live 510 V bus and sits in FAULT_UV_WAIT	wrong V_DC_SENSED_OFFSET for that board variant	wrong image for the board. Check which variant it is

Tuning

symptom	cause
fault 44, rotor did not turn	check the plate first. The gate is 566 / plate_I_rms mH, so a plate current entered *too low raises* the gate
tune fails only on PV, passes on a stiff supply	not enough watts. Tune on the stiffest supply available
tune passes but disagrees with the plate (47)	it may not be the motor you think it is
sigma-Ls looks too high	dead time. It stays at the shared 188 — under-compensating inflates apparent sigma-Ls and turns a marginal tune into a reliable 44

Flashing

symptom	cause	fix
OTA stalls part way	an old bootloader (CAB6EC2A) cannot be repaired over serial	SWD recovery. Check which bootloader the drive runs before blaming the cable
every app "dies at boot" after a bootloader flash	a Debug bootloader was flashed — 4260 B into a 4096 B slot, overrunning the app region	flash the -0s Release bootloader, 3268 B, from boot loader /
new app refuses to boot after SWD	stale OTA metadata: it carries the *old* image's length and CRC	mass erase first; leave 0x0800F800 erased
random OTA corruption over RS485	an auto-direction adapter clipping the last byte	fixed in bootloader 18274D72 with a pad byte, reply lead and per-chunk CRC. Check the bootloader version
FLASH.bat refuses the board	no STABLE image for it — nobody has proven one	ask the engineer. Do not flash LATEST on your own judgement

Settings that will not stick

The usual cause is no SAVE_SETTINGS — a write is RAM only and dies at the next reset, and these boards reset often. One board showed 104 resets.

It looks exactly like the drive "reverting" a good write. It is not. Save, then RELOAD_EEPROM, then read back.

Things that look like faults and are not

- STOP_REASON **11** — somebody sent a stop over the link. Including a tool.
- STOP_REASON **12 / 13** — deliberate probes, not failures.
- BUS_SHED dipping on a cloud edge — the droop doing its job.
- FOC_TUNE_STS reading 33 — it is **packed**, not a plain flag: $\text{tuned} + 2 \times \text{fail} + 4 \times (\text{state} \& 0xF) + 64 \times (\text{failcode} \& 0xF)$. A healthy tune parked at the last stage reads $1 + 4 \times 8 = 33$.
- MEAS_* registers reading 0 while TEMP, VDC, IDC are live — that mirror group is unpopulated. Cosmetic.
- MOD_IDX reading a flat 0.2000 — also unpopulated. Use FOC_VDEM against $V_{dc}/\sqrt{3}$ instead.

When to stop and ask

- OC_INST (23) — likely hardware, not load.
- INSUL_LEAK (30) — megger the motor and cable **before** restarting. There is a wet joint 70 m down until proven otherwise.
- An OTA that stalled part way.
- Anything where the fix would mean changing a protection limit.

Fault and stop codes

Read `FAULT_DETAIL` and `STOP_REASON` from the dashboard (`DASHBOARD.bat`) or with `foc_diag.py`. STS gives the coarse state; `FAULT_DETAIL` says *why*.

Codes come from `drive/hs300_drive.h`. If a code here disagrees with the firmware, the firmware is right — say so and this file gets fixed.

Drive state — STS

value	state	meaning
0	OFF	stopped, gates off
1	ON	running
2	FAULT	stopped by a fault; read <code>FAULT_DETAIL</code>
3	FAULT_OC	over-current
4	FAULT_OV	over-voltage
5	FAULT_UV	under-voltage
6	FAULT_UV_WAIT	waiting for the bus to come back
7	FAULT_OT	over-temperature

MODE: 0 = MANUAL, 1 = MPPT, 2 = DCBUS.

Over-current — 20 to 23

All four mean current, but not the same current, and the action differs.

code	name	what tripped	first thing to check
20	OC_ENVELOPE	$id^2 + iq^2$ over the envelope	a real overload — pump jammed, sand, a dry well
21	OC_TRACKING	the current loop lost authority	the motor is not following: wrong tune, a weak bus, a failing phase
22	OC_RMS_TOC	RMS time-overcurrent, the thermal one	sustained overload rather than a spike. Check F01 against the plate
23	OC_INST	instantaneous per-phase	a short or a shoot-through. Do not just restart — look for a fault in the power stage or the motor leads

23 is the serious one. 20 and 22 usually mean the load; 23 usually means hardware.

Insulation and winding — 30 to 32

code	name	meaning	action
30	INSUL_LEAK	zero-sequence current — current going to earth	a wet joint or a damaged cable, 70 m down. Megger the motor and the cable before restarting
31	WINDING_ASYM	the three line-to-line resistances disagree	see the warning below before believing it

code	name	meaning	action
32	PHASE_OPEN	one lead broken, the other two intact	PS_FLAGS names which phase. Check the joints at the head first

31 is often not what it says

Read PS_FLAGS before trusting a 31. If bit 0x4000 (PS_F_OVERCUR) is set, a probe vector aborted on over-current and left *no measurement behind* — that is not asymmetry at all. Fix the over-current cause and re-run.

A relay or an ADC-offset problem can also fake an open phase. That exact chain cost a day on the grid board on 2026-09-19.

Failed auto-tune — 40 to 48

40 + failcode. The drive is stopped and **not tuned**; it will not run closed loop until a tune succeeds.

code	meaning	usual cause
41	Rs measurement failed	no motor, an open phase, or current sensing wrong
42	the 40 Hz injection failed	could not drive enough current — weak bus, or F01 too low
43	leakage inductance out of band	wrong plate, or the measurement is being clipped
44	the rotor did not turn	the most common. See below
45	the rotor dragged	something mechanical, or far too little bus
46	could not hold current	the bus sagged during the tune — tune on a stiff supply
47	the result disagrees with the nameplate	the plate is wrong, or it is not the motor you think
48	current abort	hit the limit; F01 or the plate is wrong

44 — the rotor did not turn

The gate behind 44 is derived from the **plate**, as $566 / \text{plate_I_rms}$ mH. So a plate current entered **too low RAISES the gate** and refuses a perfectly good spin. Check the plate before touching anything else.

On the 22 kW well machine PLATE_I_RMS is deliberately **45 A**, not the 39 A nameplate — that is intentional and documented. Do not "correct" it.

Tune on the stiffest supply you have. On PV at dawn there is not enough power and you will chase a fault that is really a lack of watts.

Clean stops that are not faults — 11 to 13

code	meaning
11	a STOP arrived over the serial link (a tool or the dashboard)
12	START_DEFER — the bus could not carry the start, so the drive did not fight it
13	LINGER — it was running below pump cut-in on a falling sun and gave up

12 and 13 are correct behaviour, not failures. They replaced a dawn pattern of ten UV trips in a row: the I-f start needs roughly 750 W of magnetising current and the array had 50–800 W. Probing and deferring is the fix.

Pre-start check — PS_FLAGS bits

Run before every start unless PRESTART_EN is 0.

bit	name	severity
0x01	SKIPPED	disabled by PRESTART_EN
0x02	NO_MOTOR	warn — all three pairs open, nothing connected
0x04	R_HIGH	warn — symmetric but high against the stored Rs
0x08	LEAK	fail — zero-sequence current
0x10	ASYM	fail — spread over the limit
0x20	OPEN	fail — one or two pairs open
0x40 / 0x80 / 0x100	PH_U / PH_V / PH_W	which phase the open pairs share
0x200 / 0x400 / 0x800	RAIL_UV / RAIL_VW / RAIL_WU	that vector railed the ADC
0x4000	OVERCUR	a vector aborted on over-current — no measurement was taken

PS_STATE: 7 = pass, 8 = failed. PS_VCODE packs four bits per line-to-line vector; 5 means that vector hit over-current.

A start within about 2 seconds of a stop fails the pre-start on a coasting motor — the back-EMF looks like a fault. Wait, then start.

Serial link — communications guide

The drive speaks a small binary protocol over serial. `tools/hs300_protocol.py` is the **authority**: if this guide and that file disagree, the file is right.

Physical layer

baud	per drive - read BAUD_COM2. This plant's well drive is 9600
interface	RS485 (USB-RS485 adapter) or TTL (the ST-Link's USB serial)
framing	8N1
CRC	CRC16-CCITT, poly 0x1021, init 0xFFFF , over CMD..payload

`crc16("123456789")` must give `0x29B1` — use that to check any new implementation. **It is not the Modbus CRC**; the flow meter in `modbus_rtu.py` uses a different one. Mixing them up produces frames that are rejected without explanation.

The baud is a per-drive setting, not a constant. It lives in EEPROM as `BAUD_COM2` (and `BAUD_COM1`), an index into `{2400, 4800, 9600, 19200, 38400, 57600, 115200}`. The well drive here reads `BAUD_COM2 = 2`, which is **9600**.

This line used to say "9600 does not work on these builds, use 115200". That was wrong, and it cost an evening on 2026-10-03: every tool defaulted to 115200, every read came back as an ambiguous silence, and the blame went to the cable, the adapter, the bridge board and the drive's own power before anyone checked the baud.

You should not have to know. `open(verify=True)` pings for the firmware magic and, if nothing answers, scans the rest of the table and carries on at whatever replies, saying so. Every tool does this now. If a tool reports no drive, it has already tried every baud — believe it, and go and look at the drive.

In Python the port is opened as:

```
import hs300_protocol as P
L = P.Link('COM4', baud=115200, mode='rs485') # or mode='ttl'
L.open(verify=True) # pings, scans the bauds, raises if nothing is there

L = P.open_drive('COM4') # or just this: finds the baud by itself
```

Two register banks

bank	count	access
telemetry	167	read only — what the drive is doing
settings	75	read/write — how it is configured

Look settings up BY NAME, never by a remembered index:

```
L.read_telemetry() # the whole bank
L.read_settings()
L.write_setting(P.SET_BY_NAME['F09 FMAX'], 50.0)
L.command('STOP')
```

`P.SET_BY_NAME` and `P.TEL_BY_NAME` are the maps. Settings travel in **real units** over the wire — the panel's F-number and its $\times 10$ scaling are a separate thing, and confusing the two has caused real mistakes.

Telemetry is a snapshot

`comm_publish()` copies the live values into the register bank **once per 404 ms** UI slot. Polling faster returns the same numbers. 500 ms costs nothing and gains nothing over 400 ms.

So a value that looks frozen for a few hundred milliseconds is normal. A value frozen for **seconds** is not.

Writes do not persist by themselves

`write_setting()` changes the live value and the settings mirror. It does **not** touch the EEPROM. `CMD_SAVE_SETTINGS` is, in the firmware's own words, the only thing that writes it — 64 bytes, about 20 ms of blocked main loop, deliberately an explicit action.

To prove a value is really stored:

```
write_setting(...) -> CMD_SAVE_SETTINGS -> CMD_RELOAD_EEPROM -> read back
```

Reading straight after the write proves nothing.

The register map must match the firmware

If a telemetry register moves and the PC-side map does not, **the tools misread the drive silently** — no error, just wrong numbers. `COMM_FW_MAGIC` is bumped when the map changes so a stale tool is refused rather than believed.

There is no error for a *tool* that is behind. That is why `tools/` here is copied from the firmware tree on every publish instead of maintained separately: a second hand-kept copy once sat three map revisions behind and was blind to every register above 103.

`check_link_api.py` resolves every call every tool makes against the current map. `FAILURES: 0` means nothing is broken.

The capture recorder

The drive keeps a ring buffer — 3 channels × 256 samples — running from boot. Any protection trip freezes it, so the 51 ms leading *into* the trip is on the chip without anyone having armed anything.

```
python tools/hs300_capture_dump.py COM4 --no-arm --out trip.csv
```

Channels are selected by source id; `CAP_DIV` decimates, and the window is $256 \times \text{CAP_DIV} / 5000$ seconds.

Decimation is raw, not averaged, so anything above $2500/\text{CAP_DIV}$ Hz aliases into the record — fine for fast questions at `CAP_DIV = 1`, a trap for slow ones.

Two sources worth knowing, because the obvious ones mislead:

- 31 `vdc_inst_v` and 32 `idc_inst_a` are the **unaveraged** DC pair. VDC/IDC come from the RMS path and only update once per ~80 ms block, so a fast capture of those **stair-steps** — one trace showed 158 identical samples in a row — and any sample taken while the bus is moving is an average across a changing operating point.
- 12 `vDcInst` exists but is stored ×100 in a uint16, so it **clips at 655 V**. Useless on a bus that runs at 650–750 V. Use 31.

MQTT and the relay box

Separate device, separate repo. It uses the public broker `.hivemq.com` with the topic prefix `relay/<DEVICE_ID>/`.

There is no authentication on that path — the device id is the whole access control. Treat it as such.